

LAPORAN PRATIUM STRUKTUR DATA

IMPLEMENTASI STACK DAN PRINSIP LIFO DALAM JAVA

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr.WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum "Implementasi Stack dan Prinsip LIFO dalam Java" ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Stack yang bekerja dengan prinsip LIFO (Last In, First Out), serta penerapannya melalui implementasi menggunakan library bawaan Java, ArrayList, array statis, dan studi kasus nyata seperti evaluasi notasi postfix dan simulasi riwayat browser. Saya menyadari bahwa pemahaman tentang struktur data dengan akses terbatas merupakan fondasi penting dalam algoritma rekursi, manajemen memori, dan pengembangan aplikasi yang efisien.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya supaya laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	4
BAB II PEMBAHASAN.....	5
2.1 Konsep Stack dalam Java.....	5
2.2 Langkah pengerjaan	6
2.3 Latihan	10
BAB III KESIMPULAN.....	13
3.1 Kesimpulan	13
3.2 Saran	13
DAFTAR PUSTAKA.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam struktur data, tidak semua koleksi data memerlukan akses acak atau penghapusan di posisi sembarang. Banyak kasus pemrograman yang justru membutuhkan aturan akses yang ketat, di mana elemen yang terakhir dimasukkan harus diproses lebih dahulu. Prinsip ini dikenal sebagai LIFO (Last In, First Out) dan diimplementasikan melalui struktur data Stack (Tumpukan).

Berbeda dengan ArrayList pada praktikum sebelumnya yang menawarkan fleksibilitas tinggi dan akses berbasis indeks, Stack membatasi operasi hanya pada satu ujung yang disebut top. Pembatasan ini justru memberikan keunggulan dalam hal efisiensi, keamanan data, dan kemudahan implementasi pada masalah tertentu seperti navigasi mundur, pembatalan aksi (undo), evaluasi ekspresi matematika, serta manajemen panggilan fungsi (call stack). Pada praktikum ini, mahasiswa diminta mengimplementasikan Stack melalui berbagai pendekatan (library, ADT berbasis ArrayList, dan array statis) serta menerapkannya pada studi kasus nyata.

1.2 Tujuan

1. Memahami prinsip kerja struktur data Stack dan konsep LIFO.
2. Mengimplementasikan Stack menggunakan class bawaan Java `java.util.Stack`.
3. Membangun ADT Stack secara manual menggunakan ArrayList dan array statis.
4. Menerapkan Stack untuk menyelesaikan masalah algoritma seperti evaluasi notasi postfix dan simulasi riwayat navigasi.
5. Menganalisis kompleksitas waktu dan ruang dari setiap operasi dasar Stack.

1.3 Manfaat

1. Mahasiswa mampu membedakan karakteristik Stack dengan struktur data linier lainnya seperti Array dan ArrayList.
2. Mahasiswa memahami pentingnya pembatasan akses data dalam merancang sistem yang lebih aman dan terprediksi.
3. Mahasiswa memperoleh pengalaman dalam menerjemahkan algoritma matematis (postfix) dan logika aplikasi (browser history) ke dalam implementasi kode yang efisien.
4. Melatih kemampuan debugging dan penanganan kondisi batas seperti Stack Overflow dan Stack Underflow.

BAB II

PEMBAHASAN

2.1 Konsep Stack dalam Java

Stack adalah koleksi objek yang terurut secara linier dengan aturan penyisipan dan penghapusan yang hanya dilakukan pada satu ujung (top). Operasi fundamental pada Stack meliputi:

- `push(e)`: Menambahkan elemen `e` ke puncak stack.
- `pop()`: Mengambil dan menghapus elemen teratas.
- `peek()` / `top()`: Melihat elemen teratas tanpa menghapusnya.
- `isEmpty()`: Memeriksa apakah stack kosong.
- `size()`: Mengembalikan jumlah elemen.

Dalam Java, Stack dapat diimplementasikan melalui:

1. `java.util.Stack` (legacy class, thread-safe, namun lebih lambat).
2. Implementasi manual menggunakan Array Statis atau `ArrayList/LinkedList` untuk memahami mekanisme internal.

2.2 Langkah pengerjaan

1. NilaiMaksimum_2511532029

Program:

```
1 package pekan3_25115302029;
2
3 import java.util.Stack;
4
5 public class NilaiMaksimum_2511532029 {
6     public static int max_2029(Stack<Integer> s_2029){
7         Stack<Integer> backup_2029 = new Stack<Integer>();
8         int maxValue_2029 = s_2029.pop();
9         backup_2029.push(maxValue_2029);
10        while (!s_2029.isEmpty()){
11            int next_2029 = s_2029.pop();
12            backup_2029.push(next_2029);
13            maxValue_2029 = Math.max(maxValue_2029,next_2029);
14        }
15        while (!backup_2029.isEmpty()) {
16            s_2029.push(backup_2029.pop());
17        }
18        return maxValue_2029;
19    }
20    public static void main(String[] args){
21        Stack<Integer> s_2029 = new Stack<Integer>();
22        s_2029.push(70);
23        s_2029.push(12);
24        s_2029.push(20);
25        System.out.println("isi stack " + s_2029);
26        System.out.println("Stack Teratas " + s_2029.peek());
27        System.out.println("Nilai maksimum " + max_2029(s_2029));
28    }
29 }
```

Gambar 2.1

Penjelasan program:

- 1) `Stack<Integer> backup_2029 = new Stack<>();` → Membuat stack bantu untuk menyimpan elemen sementara agar stack utama tidak rusak setelah operasi.
- 2) `int maxValue_2029 = s_2029.pop();` → Inisialisasi nilai maksimum dengan elemen teratas pertama.
- 3) `while (!s_2029.isEmpty()) { ... }` → Melakukan iterasi hingga stack kosong. Setiap elemen di-pop(), dibandingkan dengan maxValue menggunakan `Math.max()`, dan disimpan ke `backup_2029`.
- 4) `while (!backup_2029.isEmpty()) { s_2029.push(backup_2029.pop()); }` → Mengembalikan seluruh elemen ke stack utama sehingga keadaan awal tetap utuh.
- 5) Analisis Kompleksitas: Waktu $O(n)$ karena setiap elemen diakses dua kali (ke backup dan kembali). Ruang $O(n)$ untuk stack bantu.

Output :

```
<terminated> NilaiMaksimum_2511532029
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 2.2

2. Pembuatan kelas Siswa_2511532029 Program:

```

1 package pekan3_2511532029;
2 import java.util.ArrayList;
3
4 class Siswa_2511532029 {
5     String nama_2029;
6     int nim_2029;
7
8     public Siswa_2511532029 (String nama_2029, int nim_2029) {
9         this.nama_2029 = nama_2029;
10        this.nim_2029 = nim_2029;
11    }
12    @Override
13    public String toString() {
14        return "Nim: " + nim_2029 + ", Nama: " + nama_2029;
15    }
16 }
17
18 public class SiswaStack_2511532029 {
19     private ArrayList<Siswa_2511532029> stack_2029;
20
21     public SiswaStack_2511532029() {
22         stack_2029 = new ArrayList<>();
23     }
24
25     public void push_2029 (Siswa_2511532029 mhs) {
26         stack_2029.add(mhs);
27     }
28
29     public Siswa_2511532029 pop_2029() {
30         if (!isEmpty()) {
31             return stack_2029.remove(stack_2029.size() - 1);
32         }
33         return null;
34     }
35
36     public Siswa_2511532029 peek_2029 () {
37         if (!isEmpty()) {
38             return stack_2029.get(stack_2029.size() - 1);
39         }
40         return null;
41     }
42
43     public boolean isEmpty() {
44         return stack_2029.isEmpty();
45     }
46
47     public void tampilkanSiswa() {
48         for (int i = stack_2029.size() - 1; i >= 0; i--) {
49             System.out.println(stack_2029.get(i));
50         }
51     }
52
53     public static void main (String[] args) {
54         SiswaStack_2511532029 studentStack_2029 = new SiswaStack_2511532029();
55
56         Siswa_2511532029 mhs1 = new Siswa_2511532029("ali",1);
57         Siswa_2511532029 mhs2 = new Siswa_2511532029("boby",2);
58         Siswa_2511532029 mhs3 = new Siswa_2511532029("charles",3);
59
60         studentStack_2029.push_2029(mhs1);
61         studentStack_2029.push_2029(mhs2);
62         studentStack_2029.push_2029(mhs3);
63
64         System.out.println("siswa di dalam stack");
65         studentStack_2029.tampilkanSiswa();
66
67         System.out.println("siswa teratas" + studentStack_2029.peek_2029());
68         System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack_2029.pop_2029());
69         System.out.println("daftar siswa setelah di pop : ");
70         studentStack_2029.tampilkanSiswa();
71     }
72 }

```

Gambar 2.3

Penjelasan program:

- 1) private ArrayList<Siswa_2511532029> stack_2029; → Menggunakan ArrayList sebagai backbone penyimpanan, namun operasi dibatasi hanya di indeks terakhir (size()-1).
- 2) push_2029() memanggil stack_2029.add(mhs); → Menambahkan elemen di akhir list (top of stack).
- 3) pop_2029() memanggil stack_2029.remove(size()-1); → Menghapus dan mengembalikan elemen terakhir.
- 4) peek_2029() memanggil stack_2029.get(size()-1); → Melihat elemen terakhir tanpa menghapus.
- 5) tampilkanSiswa() melakukan iterasi mundur (i >= 0) agar output visual sesuai dengan konsep tumpukan (top di atas).
- 6) Program mendemonstrasikan prinsip ADT (Abstract Data Type): antarmuka publik (push, pop, peek) menyembunyikan implementasi internal ArrayList.

Output:

```

<terminated> SiswaStack 2511532029 [Java Application] C:\User:
SiswaStack 2511532029 [Java Application] C:\Users\USER\p2\pool\plugins\
Nim: 3, Nama: charles
Nim: 2, Nama: boby
Nim: 1, Nama: ali
siswa teratasNim: 3, Nama: charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: charles
daftar siswa setelah di pop :
Nim: 2, Nama: boby
Nim: 1. Nama: ali

```


3. StackArray_2511532029 & StackArrayDriver_2511532029

Program:

```
1 package pekan3_2511532029;
2
3 public class stackArray_2511532029 {
4     static final int MAX_2029 = 1000;
5     int top_2029;
6     int a_2029[] = new int [MAX_2029];
7     boolean isEmpty_2029() {
8         return (top_2029 < 0);
9     }
10    stackArray_2511532029(){
11        top_2029 = -1;
12    }
13    boolean push_2029 (int x) {
14        if (top_2029 >= (MAX_2029 - 1 )) {
15            System.out.println("Stack Overflow");
16            return false;
17        }
18        else {
19            a_2029[++top_2029] = x;
20            System.out.println(x + "dimasukan data stack");
21            return true;
22        }
23    }
24    int pop_2029 () {
25        if (top_2029 < 0) {
26            System.out.println("Stack underflow");
27            return 0;
28        }
29        else {
30            int x_2029 = a_2029[top_2029--];
31            return x_2029;
32        }
33    }
34    int peek_2029 () {
35        if (top_2029 < 0) {
36            System.out.println("Stack underflow");
37            return 0;
38        }
39        else {
40            int x_2029 = a_2029[top_2029];
41            return x_2029;
42        }
43    }
44    void print_2029 () {
45        for (int i = top_2029; i > -1; i--) {
46            System.out.print(" " + a_2029[i]);
47        }
48    }
49 }
50
```

Gambar 2.4

```
1 package pekan3_2511532029;
2
3 public class stackArrayDriver_2511532029 {
4     public static void main (String[] args) {
5         stackArray_2511532029 s = new stackArray_2511532029();
6         s.push_2029(10);
7         s.push_2029(20);
8         s.push_2029(30);
9         System.out.println(s.pop_2029() + " dikeluarkan dari stack ");
10        System.out.println("dikeluarkan dari stack " + s.peek_2029());
11        System.out.print("elemen pada stack");
12        s.print_2029();
13    }
14 }
15
```

Gambar 2.5

Penjelasan program:

- 1) static final int MAX_2029 = 1000; & int a_2029[] → Implementasi berbasis array statis dengan kapasitas tetap.
- 2) int top_2029 = -1; → Inisialisasi penunjuk top. Nilai -1 menandakan stack kosong.
- 3) push_2029(int x):
- 4) Cek if (top_2029 >= MAX_2029 - 1) → Mencegah *Stack Overflow*.
- 5) a_2029[++top_2029] = x; → Increment pointer terlebih dahulu, baru isi array.
- 6) pop_2029():
- 7) Cek if (top_2029 < 0) → Mencegah *Stack Underflow*.
- 8) return a_2029[top_2029--]; → Kembalikan nilai, lalu decrement pointer.
- 9) Driver (stackArrayDriver_2029) menguji urutan LIFO: push 10, 20, 30 → pop mengembalikan 30 → peek menampilkan 20.

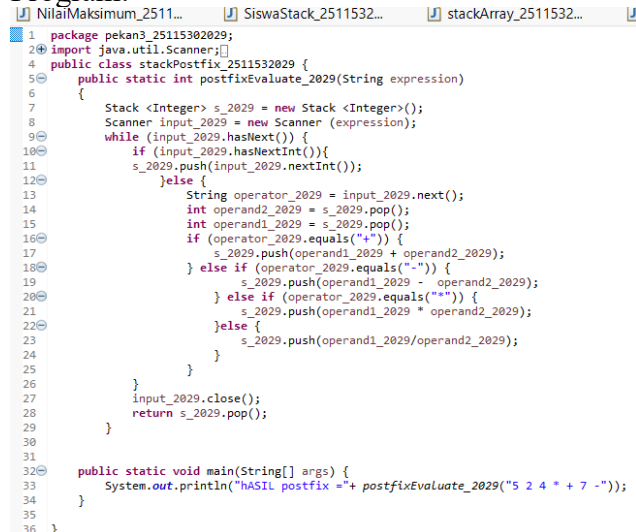
Output:

```
<terminated> stackArrayDriver_2511
10dimasukan data stack
20dimasukan data stack
30dimasukan data stack
30 dikeluarkan dari stack
dikeluarkan dari stack 20
elemen pada stack 20 10
```

Gambar 2.6

4. stackPostfix_2511532029

Program:



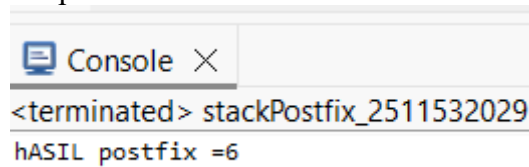
```
1 package pekan3_25115302029;
2 import java.util.Scanner;
3
4 public class stackPostfix_2511532029 {
5     public static int postfixEvaluate_2029(String expression)
6     {
7         Stack<Integer> s_2029 = new Stack<Integer>();
8         Scanner input_2029 = new Scanner(expression);
9         while (input_2029.hasNext()) {
10             if (input_2029.hasNextInt()) {
11                 s_2029.push(input_2029.nextInt());
12             } else {
13                 String operator_2029 = input_2029.next();
14                 int operand2_2029 = s_2029.pop();
15                 int operand1_2029 = s_2029.pop();
16                 if (operator_2029.equals("+")) {
17                     s_2029.push(operand1_2029 + operand2_2029);
18                 } else if (operator_2029.equals("-")) {
19                     s_2029.push(operand1_2029 - operand2_2029);
20                 } else if (operator_2029.equals("*")) {
21                     s_2029.push(operand1_2029 * operand2_2029);
22                 } else {
23                     s_2029.push(operand1_2029/operand2_2029);
24                 }
25             }
26         }
27         input_2029.close();
28         return s_2029.pop();
29     }
30
31     public static void main(String[] args) {
32         System.out.println("HASIL postfix =" + postfixEvaluate_2029("5 2 4 * + 7 -"));
33     }
34 }
35
36 }
```

Gambar 2.7

Penjelasan program:

- 1) Scanner input_2029 = new Scanner(expression); → Memisahkan token ekspresi berdasarkan spasi.
- 2) if (input_2029.hasNextInt()) s_2029.push(...); → Jika token angka, langsung di-push ke stack.
- 3) else { ... } → Jika token operator (+, -, *, /):
operand2_2029 = s_2029.pop(); → Ambil operand kanan terlebih dahulu.
operand1_2029 = s_2029.pop(); → Ambil operand kiri.
Lakukan operasi aritmatika sesuai operator, lalu push hasilnya kembali.
- 4) Trace Ekspresi "5 2 4 * + 7 -":
Push: 5, 2, 4
* → pop 4 & 2 → 2*4=8 → push 8
+ → pop 8 & 5 → 5+8=13 → push 13
Push: 7
- → pop 7 & 13 → 13-7=6 → push 6
return s_2029.pop(); → Mengembalikan 6.

Output:



Gambar 2.8

2.3 Latihan

Deskripsi Program:

Program mensimulasikan fitur navigasi browser menggunakan prinsip LIFO. Setiap halaman yang dikunjungi disimpan di stack, dan tombol "Back" menghapus halaman teratas

1.Program Website_2511532029

```
1 package pekan3_25115302029;
2
3 public class Website_2511532029 {
4     private String judul_2029;
5     private String url_2029;
6
7     public Website_2511532029(String judul_2029, String url_2029) {
8         this.judul_2029 = judul_2029;
9         this.url_2029 = url_2029;
10    }
11
12    public String getJudul_2029() { return judul_2029; }
13    public String getUrl_2029() { return url_2029; }
14
15    public void setJudul_2029(String judul_2029) { this.judul_2029 = judul_2029; }
16    public void setUrl_2029(String url_2029) { this.url_2029 = url_2029; }
17
18    @Override
19    public String toString() {
20        return "Judul: " + judul_2029 + "\nURL : " + url_2029;
21    }
22 }
```

Gambar 2.10

a) Deskripsi program Website_2511532029 (ADT Data)

- Berisi atribut judul_2029 dan url_2029 dengan encapsulasi private.
- Menyediakan getter, setter, dan override toString() untuk format tampilan.

3. Program Playlist_2511532029

```
1 package pekan3_2511532029;
2 import java.util.Stack;
3 import java.util.Scanner;
4 public class Browser_2511532029 {
5     public static void main(String[] args) {
6         Scanner scanner = new Scanner(System.in);
7         Stack<Website_2511532029> history_2029 = new Stack<>();
8         int pilihan;
9         do {
10             System.out.println("\n=== Browser History NIM: 2511532029 ===");
11             System.out.println("1. Kunjungi Website ");
12             System.out.println("2. Tombol Back ");
13             System.out.println("3. Lihat Halaman Aktif");
14             System.out.println("4. Cek Status History");
15             System.out.println("5. Keluar");
16             System.out.print("Pilihan: ");
17             pilihan = scanner.nextInt();
18             scanner.nextLine();
19             switch (pilihan) {
20                 case 1:
21                     System.out.print("Masukkan Judul: ");
22                     String judul_2029 = scanner.nextLine();
23                     System.out.print("Masukkan URL: ");
24                     String url_2029 = scanner.nextLine();
25
26                     history_2029.push(new Website_2511532029(judul_2029, url_2029));
27                     System.out.println("Berhasil mengunjungi halaman!");
28                     break;
29                 case 2:
30                     if (history_2029.isEmpty()) {
31                         System.out.println("History kosong. Tidak ada halaman untuk dikembalikan.");
32                     } else {
33                         Website_2511532029 lastSite = history_2029.pop();
34                         System.out.println("Berhasil kembali. Halaman yang dihapus: " + lastSite.getJudul_2029());
35                     }
36                     break;
37                 case 3:
38                     if (history_2029.isEmpty()) {
39                         System.out.println("History kosong. Tidak ada halaman aktif.");
40                     } else {
41                         Website_2511532029 currentSite = history_2029.peek();
42                         System.out.println("\n--- Halaman Aktif ---");
43                         System.out.println(currentSite.toString());
44                     }
45                     break;
46                 case 4:
47                     System.out.println("Status History: " + (history_2029.isEmpty() ? "Kosong" : "Terdapat Data"));
48                     System.out.println("Jumlah total riwayat tersimpan: " + history_2029.size());
49                     break;
50                 case 5:
51                     System.out.println("Keluar dari program...");
52                     break;
53                 default:
54                     System.out.println("Pilihan tidak valid.");
55                     break;
56             }
57         } while (pilihan != 5);
58         scanner.close();
59     }
60 }
```

Gambar 2,11

a) Deskripsi program Browser_2511532029(Logika Aplikasi)

- Stack<Website_2511532029> history_2029 → Struktur data utama.
- case 1 (Kunjungi Website): Membuat objek Website baru dan melakukan push(). Halaman terbaru otomatis menjadi top.
- case 2 (Tombol Back): Melakukan pengecekan isEmpty(). Jika tidak kosong, pop() menghapus halaman terakhir dan menampilkan konfirmasi. Jika kosong, menampilkan pesan error yang aman.
- case 3 (Lihat Halaman Aktif): Menggunakan peek() untuk menampilkan halaman teratas tanpa menghapusnya.
- case 4 (Cek Status): Menampilkan status kosong/terisi dan jumlah riwayat dengan history_2029.size().
- do-while + switch → Membangun antarmuka konsol interaktif yang terus berjalan hingga user memilih keluar.

Output:

```
<terminated> Browser_2511532029 [Java Application] C:\Use
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511532029 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilihan: 3

--- Halaman Aktif ---
Judul: SSO UNAND
URL : https://sso.unand.ac.id/login

=== Browser History NIM: 2511532029 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilihan: 4
Status History: Terdapat Data
Jumlah total riwayat tersimpan: 1

=== Browser History NIM: 2511532029 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilihan: 2
Berhasil kembali. Halaman yang dihapus: SSO UNAND

=== Browser History NIM: 2511532029 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilihan: 4
Status History: Kosong
Jumlah total riwayat tersimpan: 0

=== Browser History NIM: 2511532029 ===
1. Kunjungi Website
2. Tombol Back
3. Lihat Halaman Aktif
4. Cek Status History
5. Keluar
Pilihan: 5
Keluar dari program...
```

Gambar 2.11

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan pelaksanaan praktikum implementasi Stack dalam Java, dapat disimpulkan bahwa Stack merupakan struktur data linier yang bekerja dengan prinsip LIFO (Last In, First Out), di mana seluruh operasi manipulasi data difokuskan hanya pada elemen teratas atau top. Struktur ini dapat diimplementasikan melalui berbagai pendekatan, mulai dari class bawaan `java.util.Stack`, pembungkusan `ArrayList` untuk pengelolaan kapasitas dinamis, hingga penggunaan array statis yang menawarkan efisiensi alokasi memori sangat tinggi. Seluruh operasi dasar Stack seperti push, pop, peek, dan isEmpty memiliki kompleksitas waktu $O(1)$, menjadikannya sangat optimal untuk skenario yang tidak memerlukan pencarian atau penghapusan di posisi acak. Penerapan konsep Stack telah berhasil divalidasi melalui studi kasus evaluasi ekspresi notasi postfix dan simulasi riwayat navigasi browser, yang membuktikan relevansinya dalam pengembangan compiler, kalkulator ilmiah, serta antarmuka aplikasi modern. Selain itu, praktikum ini menekankan pentingnya penanganan kondisi batas seperti overflow dan underflow guna mencegah runtime exception dan menjamin stabilitas serta keandalan program secara keseluruhan.

3.2 Saran

1. Akan lebih baik bila dosen menyelenggarakan sesi pra-praktikum di kelas agar mahasiswa dapat memperoleh pemahaman awal yang lebih memadai, sehingga dapat menghindari kepanikan atau kesalahan saat praktikum
2. Sebaiknya dosen membagikan materi praktikum terlebih dahulu melalui iLearn agar mahasiswa bisa mempersiapkan diri sebelum pelaksanaan praktikum

DAFTAR PUSTAKA

- [1] Oracle. "Stack (Java Platform SE 17)". *The Java™ Tutorials*. 2024.
Available:
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Stack.html>